

CHAPTER 02

기본 문법

- | | |
|--------------|------------|
| 01 기본 용어 | 06 증감 연산자 |
| 02 출력 | 07 자료형 검사 |
| 03 기본 자료형 | 08 var 키워드 |
| 04 변수 | 09 입력 |
| 05 복합 대입 연산자 | 10 자료형 변환 |



■ 표현식과 문장

- 표현식: 다음과 같이 값을 만들어내는 간단한 코드

```
273
10 + 20 + 30 * 2
"C# Programming"
```

- 문장: 표현식이 하나 이상 모이고, 마지막에 세미콜론이 찍힌 것. 코드 문장 마지막엔 종결의 의미로 세미콜론을 찍음

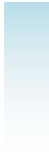
```
273;
10 + 20 + 30 + 2;
var name = "윤" + "인" + "성"
Console.Write("Hello C# Programming");
```

■ 키워드

- 키워드: 특별한 의미가 부여된 단어. C# 키워드는 다음과 같음

표 2-1 일반 키워드

abstract	as	base	bool	break
byte	case	catch	char	checked
class	const	continue	decimal	default
delegate	do	double	else	enum
event	explicit	extern	false	finally
fixed	float	for	foreach	goto
if	implicit	in	int	interface
internal	is	lock	long	namespace
new	null	object	operator	out
override	params	private	protected	public
readonly	ref	return	sbyte	sealed
short	sizeof	stackalloc	static	string
struct	switch	this	throw	true
try	typeof	uint	ulong	unchecked
unsafe	ushort	using	virtual	void
volatile	while			

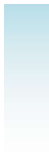


■ 키워드

- 다음 키워드는 특정 위치에서만 키워드로 작동함

표 2-2 컨텍스트 키워드(또는 문맥 키워드)

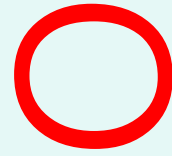
add	and	alias	ascending	args
async	await	by	descending	dynamic
equals	file	from	get	global
group	init	into	join	let
managed	nameof	nint	not	nonnull
nuint	on	or	orderby	partial
record	remove	required	scoped	select
set	unmanaged	value	var	when
where	with	yield		



■ 식별자

- 식별자: 이름을 붙일 때에 사용하는 단어. 다음과 같은 규칙을 지켜야 함
 - 키워드를 사용하면 안 됨
 - 특수 문자는 _만 허용함
 - 숫자로 시작하면 안 됨
 - 공백은 입력하면 안 됨
- 예를 들어 왼쪽의 단어는 식별자로 사용할 수 있지만 오른쪽은 사용할 수 없음

alpha
alpha10
_alpha
ALpha
ALPHA



break
273alpha
has space



■ 식별자

- 식별자를 만들 때는 알파벳을 사용하는 것이 관례
- 알파벳 외의 관례를 정리하면 다음과 같음
 - ① 클래스, 속성, 메서드, 네임스페이스의 이름은 항상 대문자로 시작한다.
 - ② 지역 변수와 전역 변수의 이름은 항상 소문자로 시작한다.
 - ③ 여러 단어로 이루어진 식별자는 각 단어의 첫 글자를 대문자로 한다.
- ③번은 식별자의 의미를 파악하려고 만든 규칙

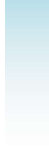
```
i love you → iLoveYou  
i am a boy → iAmABoy  
create server → createServer
```

■ 식별자

- 식별자를 구분하는 방법: 변수와 메서드
- 식별자 뒤에 괄호가 있으면 이 식별자를 메서드라고 부르고 그 이외의 것을 변수라고 부름

```
Console.WriteLine("Hello C# Programming"); //메서드
Math.PI; //변수
Math.Floor(10.1); //메서드
Console.BackgroundColor //변수
```

- 첫 번째, 세 번째 식별자(WriteLine과 Floor)는 괄호가 있으므로 메서드
- 두 번째와 네 번째 식별자 (PI와 BackgroundColor)는 변수
- 메서드의 괄호 안에 넣는 것을 매개변수(Parameter) 라고 부름

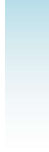


■ 주석

- 주석은 프로그램의 진행에 전혀 영향을 주지 않는 코드로, 프로그램을 선택 명하는 데 사용하는 코드
- C#가 지원하는 두 가지 주석 처리 방법
 - //을 사용해 한 줄만 주석으로 처리
 - 둘째는 /*와 */를 사용해 여러 줄을 주석으로 처리하는 방법

표 2-3 주석 처리 방법

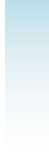
방법	표현
한 줄 주석 처리	// 주석
여러 줄 주석 처리	/*
	주석
	주석
	*/



■ 주석

- 다음 예시에서 Console.WriteLine() 메서드를 실행했지만 주석으로 처리했으므로 모두 무시됨

```
// 주석은 코드의 실행에 영향을 주지 않습니다.  
/*  
Console.WriteLine("C# Programming");  
Console.WriteLine("C# Programming");  
Console.WriteLine("C# Programming");  
*/
```



■ 출력 방법

- 방법1 : Console 클래스의 WriteLine () 메서드 사용

Console.WriteLine(출력하고_싶은_대상);

그림 2-1 Console.WriteLine() 메서드의 형태

- 기본예제 2-1 C# 기본 출력 익히기 [/2장/Output](#)

코드 2-1

Hello 예제

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("Hello C# Programming .. !");
04 }
```

실행 결과

Hello C# Programming .. !

Microsoft Visual Studio 디버그 X

Hello C# Programming .. !

D:\Workspace_C#\2장\Output\

■ 정수

- 정수: 273, 52, -103, 0처럼 소수점이 없는 숫자
- C#은 정수를 다음과 같은 방법으로 생성

코드 2-2

/2장/DefaultData

정수 생성

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(52);
04 }
```

정수

- 만든 정수는 다음과 같은 사칙 연산자로 연산할 수 있음

표 2-4 기본적인 사칙 연산자

연산자	설명
+	덧셈 연산
-	뺄셈 연산
*	곱셈 연산
/	나눗셈 연산

코드 2-3

정수 덧셈 연산자

/2장/DefaultData

```
01 static void Main(string[] args)
02 {
03     // 325를 출력합니다.
04     Console.WriteLine(52 + 273);
05 }
```

325

■ 정수

- 연산을 할 때는 연산자 우선순위를 고려

코드 2-4 연산자 우선순위

/2장/DefaultData

```
01 static void Main(string[] args)
02 {
03     // 결과를 예측해봅시다.
04     Console.WriteLine(5 + 3 * 2);
05 }
```

■ 정수

- 정수를 사용하여 나머지 연산을 할 때는 좌변을 우변으로 나눈 나머지를 표시함

표 2-5 나머지 연산자

연산자	설명
%	나머지 연산자

코드 2-5 나머지 연산자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(10 % 5);
04     Console.WriteLine(7 % 3);
05 }
```

/2장/DefaultData

01
02
03
04
05

0
1



■ 정수

- C#은 정수끼리 연산하면 결과도 정수로 나옴
- 나누었을 때에 소수점이 생기는 부분은 모두 사라짐
- 예를 들어 10/4는 2.5가 아니라 2

정수	+	정수	=	정수
정수	-	정수	=	정수
정수	*	정수	=	정수
정수	/	정수	=	정수
정수	%	정수	=	정수

그림 2-2 정수 연산 결과

■ 정수

■ 기본예제 2-2 정수와 연산자

/2장/IntegerBasic

코드 2-6 정수와 연산자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(1 + 2);
04     Console.WriteLine(1 - 2);
05     Console.WriteLine(1 * 2);
06     Console.WriteLine(1 / 2);
07     Console.WriteLine(1 % 2);
08 }
```

실행 결과

```
3
-1
2
0
1
```


■ 나머지 연산자와 부호

- 나머지 연산자의 부호는 왼쪽 피연산자의 부호를 따름

코드 2-7

음수와 나머지 연산자

/2장/DefaultData

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(4 % 3);
04     Console.WriteLine(-4 % 3);
05     Console.WriteLine(4 % -3);
06     Console.WriteLine(-4 % -3);
07 }
```

실행 결과

```
1
-1
1
-1
```

■ 실수

- 실수를 만들려면 다음과 같이 소수점(.)을 사용

코드 2-8 실수

/2장/DefaultData

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(52.273);
04 }
```

52.273

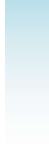
- 소수점이 들어가는 것과 안 들어가는 것은 차이가 굉장히 큼

코드 2-9 정수와 실수

/2장/DefaultData

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(0);
04     Console.WriteLine(0.0);
05 }
```

0
0



■ 실수

■ 기본예제 2-3 실수와 사칙 연산자

/2장/RealNumberBasic

코드 2-10 실수와 사칙 연산자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(1.0 + 2.0);
04     Console.WriteLine(1.0 - 2.0);
05     Console.WriteLine(1.0 * 2.0);
06     Console.WriteLine(1.0 / 2.0);
07 }
```

실행 결과

```
3
-1
2
0.5
```

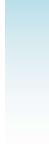
■ 문자

- 문자(Character): 글자 하나를 나타내는 자료형. 작은따옴표(')로 글자를 감싸서 만듦

'a'

그림 2-3 문자 표현

- C#은 알파벳뿐만 아니라 한국어, 중국어, 일본어, 태국어, 아랍어 등의 문자를 모두 표현할 수 있음



■ 문자

■ 기본예제 2-4 문자 /2장/CharacterBasic

코드 2-12 문자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine('A');
04     Console.WriteLine('가');
05     Console.WriteLine('나');
06 }
```

실행 결과

A
가
나

■ 문자열

- 문자열: 문자의 집합, 큰따옴표(")를 사용해 문자열을 생성
- 예: "abcdefg", "Hello World", "안녕하세요"가 문자열에 해당

코드 2-13 문자열

/2장/DefaultData

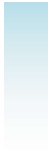
```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("안녕하세요");
04 }
```

■ 문자열

- 문자열 내에서 큰따옴표를 사용하고 싶을 때는 문자열 내부에 ₩"를 사용
- 이를 이스케이프 문자라고 부르며 특수 기능을 수행

표 2-6 자주 사용되는 이스케이프 문자

이스케이프 문자	설명	이스케이프 문자	설명
\t	수평 탭	\\	역 슬래시
\n	행 바꿈	\"	큰따옴표



■ 문자열

■ 기본예제 2-5 이스케이프 문자

/2장/EscapeCharacter

- 이스케이프 문자 중에 \t, \n, \r, \f를 사용

코드 2-14 이스케이프 문자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("한빛\ta카데미");
04     Console.WriteLine("한빛\n아카데미");
05     Console.WriteLine("\\"");
06 }
```

실행 결과

```
한빛   아카데미
한빛
아카데미
""
```


■ 문자열

- 문자열은 문자열 연결 연산자를 사용해 합칠 수도 있음

표 2-7 문자열 연결 연산자

연산자	설명
+	문자열 연결 연산자

■ 기본예제 2-6 문자열 연결 연산자

/2장/StringConnection

코드 2-15 문자열 연결 연산자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("가나다" + "라마" + "바사아" + "자차카타" + "파하");
04 }
```

실행 결과

가나다라마바사아자차카타파하

■ 문자열

■ 기본예제 2-7 문자 선택

/2장/StringSelector

표 2-8 문자 선택 괄호

연산자	설명
문자열[숫자]	문자 선택 괄호

- "안녕하세요"라는 문자열의 0번째, 1번째, 3번째에 있는 문자를 선택

코드 2-16 문자 선택

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("안녕하세요"[0]);
04     Console.WriteLine("안녕하세요"[1]);
05     Console.WriteLine("안녕하세요"[3]);
06 }
```

실행 결과

안
녕
세

■ 예외

- 문자열은 5자인데 100번째 글자를 선택하면?

코드 2-17

예외

/2장/DefaultData

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("안녕하세요"[100]);
04 }
```

- 디버그 모드에서 코드를 실행하면 다음과 같이 오류가 발생

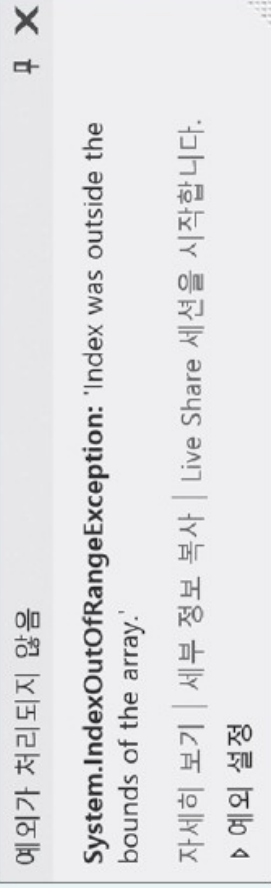


그림 2-4 예외 발생(디버그 모드)

■ 예외

- 릴리즈 모드에서 코드를 실행하면 다음과 같이 "처리되지 않은 예외"라는 오류가 나오면서 프로그램이 멈춤

실행 결과

```
Unhandled exception. System.IndexOutOfRangeException: Index was outside the
bounds of the array.
   at System.String.get_Chars(Int32 index)
   at DefaultData.Program.Main(String[] args) in C:\Users\arin\source\repos\
DefaultData\DefaultData\Program.cs:line 7

C:\Users\arin\source\repos\DefaultData\DefaultData\bin\Debug\net8.0\DefaultData.
exe(프로세스 23924개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면
자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요....
```

- 이렇게 코드를 실행하는 중에 발생하는 오류를 예외_{Exception} 또는 런타임 오류_{Runtime Error}라고 부름

■ 문자 덧셈 연산

- 문자도 연결되지 않을까 생각하고 다음과 같은 코드를 작성하는 사람도 있을 것

코드 2-18

문자 덧셈

/2장/DefaultData

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine('가' + '황');
04 }
```

- 문자는 + 연산자를 사용해도 연결되지 않음

실행 결과

99235

여기서 잠깐!

```
Console.WriteLine("가" + "행");
```



0	1	2	3	4	5
AC0	가	각	값	간	값

$$44032 + 55203 = 99235$$

$$\begin{aligned} AC00 &= (A * 16^3) + (C * 16^2) + (0 * 16^1) + (0 * 16^0) \\ &= (10 * 16^3) + (12 * 16^2) + (0 * 16^1) + (0 * 16^0) \\ &= 44032 \end{aligned}$$

$$\begin{aligned} D7A3 &= (D * 16^3) + (7 * 16^2) + (A * 16^1) + (3 * 16^0) \\ &= (13 * 16^3) + (7 * 16^2) + (10 * 16^1) + (3 * 16^0) \\ &= 55203 \end{aligned}$$

0	1	2	3
D70	헨	헨	헨
D71	헛	헛	헛
D72	힐	힐	힐
D73	힐	힐	힐
D74	훈	훈	훈
D75	흐	후	후
D76	흠	흠	흠
D77	힌	힐	힐
D78	헛	헛	헛
D79	힐	힐	힐
D7A	힐	힐	행

■ 불

- 불은 참과 거짓을 표현할 때에 사용함
- True와 false라는 두 가지 값밖에 없음

코드 2-19 불

/2장/DefaultData

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(true);
04     Console.WriteLine(false);
05 }
```

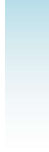
True
False

■ 불

- C# 은 크기를 비교하는 연산자로 참과 거짓을 만듭니다. 이러한 연산자를 비교 연산자라고 부름

표 2-9 비교 연산자

연산자	설명
==	같다
!=	다르다
>	왼쪽 피연산자가 크다
<	오른쪽 피연산자가 크다
>=	왼쪽 피연산자가 크거나 같다
<=	오른쪽 피연산자가 크거나 같다



■ 불

■ 기본예제 2-8 불과 비교 연산자

/2장/BoolBasic

코드 2-20

불과 비교 연산자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(52 < 273);
04     Console.WriteLine(52 > 273);
05 }
```

실행 결과

True

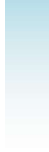
False

■ 불

- 비교 연산자를 사용하면 불을 만들 수 있고, 불끼리는 논리 연산자를 사용할 수 있음
- C#는 다음의 세 가지 논리 연산자를 제공함

표 2-10 논리 연산자

연산자	설명
!	논리 부정 연산자
	논리합 연산자
&&	논리곱 연산자



■ 불

- 논리 부정 연산자는 숫자의 부호를 반대로 만드는 - 연산자와 같은 형태로 사용
- **기본예제 2-9** 논리 부정 연산자 [/2장/LogicalNot](#)

코드 2-21

논리 부정 연산자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(!true);
04     Console.WriteLine(!false);
05     Console.WriteLine(!(52 < 273));
06     Console.WriteLine(!(52 > 273));
07 }
```

실행 결과

```
False
True
False
True
```

■ 불

논리합(||) 연산자(Logical OR):

두 개의 피연산자 중에 하나만 true이면 전체가 true

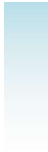
A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false



■ 불

- 논리곱(&&) 연산자 (Logical AND):
두 개의 피연산자 모두 true이어야만 전체가 true

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false



■ 불

- 논리 연산자가 가장 많이 사용되는 경우는 범위 판단과 관련된 부분
- 예를 들어 3시와 8시 사이라는 조건을 표현할 때, $3 < \text{현재 시각} < 8$ 이라고 쓰면 오류 발생.

```
int number = 10;  
Console.WriteLine(3 < number < 8);
```

CS0019 (지역 변수) int number

CS0019: '<' 연산자는 'bool' 및 'int' 형식의 피연산자에 적용할 수 없습니다.

그림 2-5 연속된 범위 연산 오류

- C#은 이러한 범위를 표현할 때 반드시 범위 연산자를 사용해야 함

■ 불

- 다음과 같은 범위는 $x < 3$ 또는 $x > 8$ 이라고 표현
- 논리합 연산자는 이렇게 나뉜 두 가지 범위를 모두 선택할 때 사용

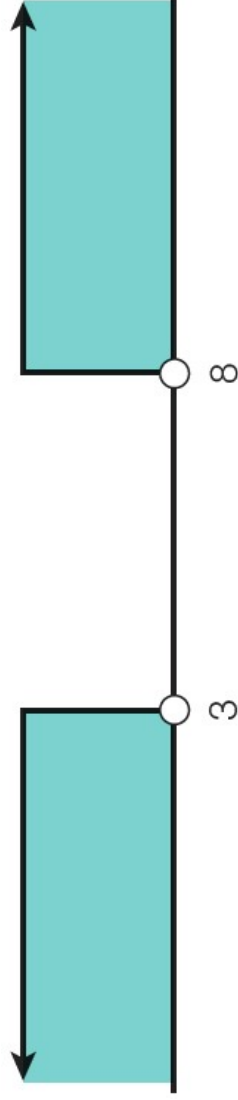


그림 2-6 $x < 3$ 또는 $8 < x$ 의 범위

- 논리곱 연산자는 다음과 같이 두 가지 범위로 한꺼번에 선택되는 부분을 찾을 때에 사용

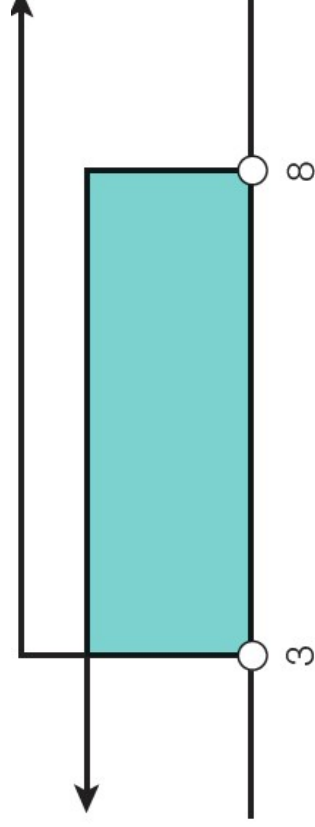


그림 2-7 $x > 3$ 그리고 $x < 8$ 의 범위

■ 불

■ 기본예제 2-10 불과 논리 연산자

/2장/LogicalOperator

코드 2-22

불과 논리 연산자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(DateTime.Now.Hour < 3 || 8 < DateTime.Now.Hour);
04     Console.WriteLine(3 < DateTime.Now.Hour && DateTime.Now.Hour < 8);
05 }
```

실행 결과

False
True

■ 변수

- 변수: 값을 저장할 때 사용하는 식별자.
- 변수는 다음 두 단계를 거쳐 사용할 수 있음
 - 변수를 선언한다.
 - 변수에 값을 할당한다.
- 변수를 만드는 것을 '변수를 선언한다'고 표현하고, 그러한 변수에 값을 넣는 것을 '변수에 값을 할당한다'라고 표현

`double pi = 3.14159265`

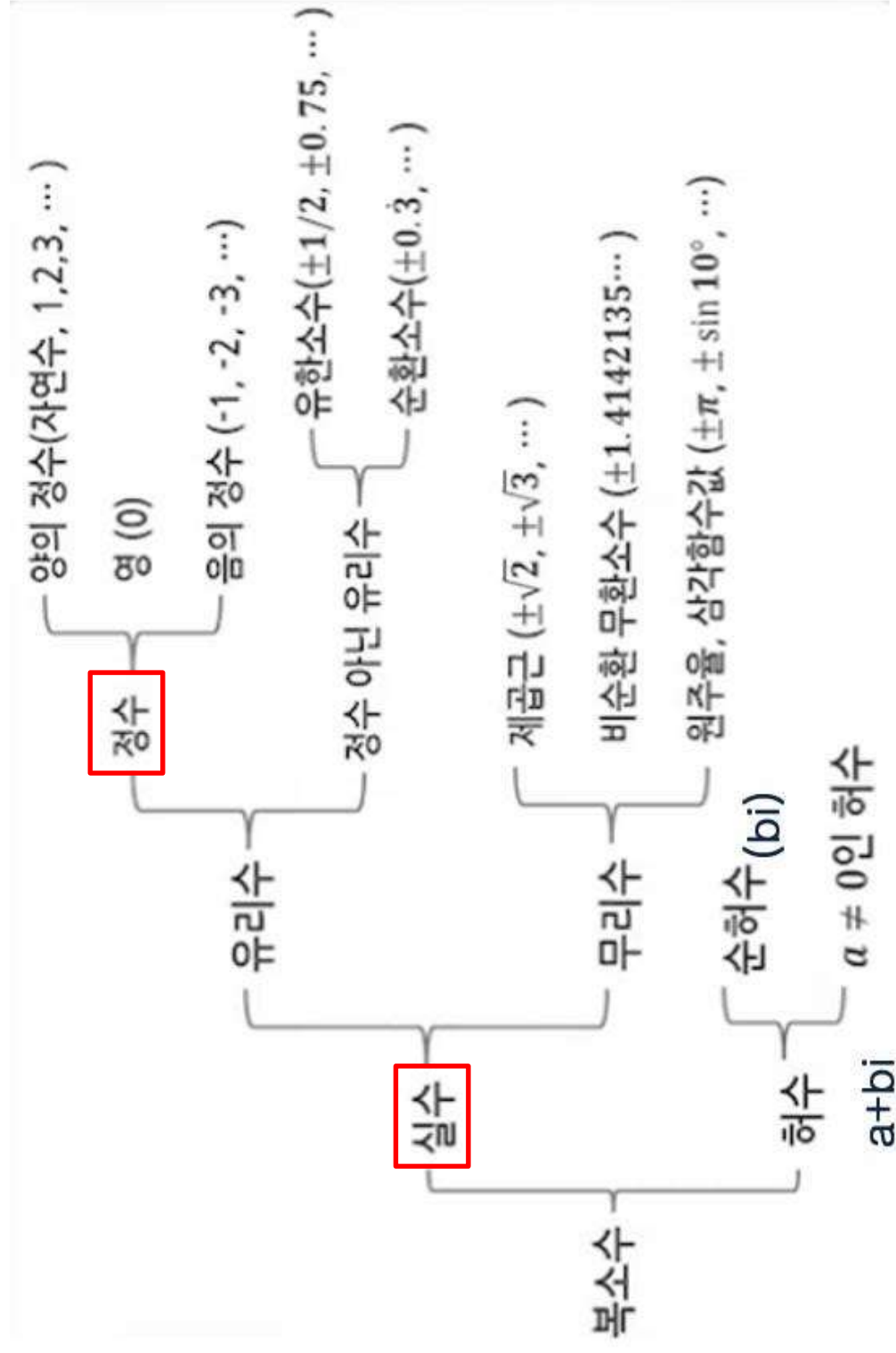
자료형

이름

값

그림 2-8 변수 선언 예

■ 정수 & 실수 자료형



■ 정수 자료형

- C#은 관련 자료형이 정말 많이 있지만 일반적으로 [표 2-13]의

정수 자료형을 가장 많이 사용

표 2-13 정수 자료형

키워드	설명
int	4바이트의 정수
long	8바이트의 정수

1 Byte = 8 Bit
4 Byte = 32 Bit
8Byte = 64 Bit
1 Bit : 0, 1 / 8 Bit = 2^8 /
32 Bit = 2^{32} / 64 Bit = 2^{64}

- 이러한 자료형을 사용해서 다음과 같이 정수 변수를 만들 수 있음

int a = 10

long b = 20

그림 2-9 정수 자료형 선언 예

■ 정수 자료형

■ 기본예제 2-11 정수 변수 생성

/2장/IntegerVariable

코드 2-23 정수 변수 생성

```
01 static void Main(string[] args)
02 {
03     int a = 273;
04     int b = 52;
05
06     Console.WriteLine(a + b);
07     Console.WriteLine(a - b);
08     Console.WriteLine(a * b);
09     Console.WriteLine(a / b); //5.25
10     Console.WriteLine(a % b);
11 }
```

실행 결과

```
325
221
14196
5
13
```

■ 정수 자료형

- 컴퓨터는 [그림 2-10]과 [그림 2-11]처럼 정수를 2진수로 나타냄

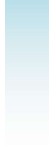
0000 0000 0000 0000 0000 0000 0000 0001

그림 2-10 1의 2진수 표현

0000 0000 0000 0000 0000 0000 0000 0100

그림 2-11 4의 2진수 표현

- 4바이트(32비트)인 int 자료형은 2^{32} 개의 숫자
(-2,147,483,648~2,147,483,647)만 나타낼 수 있음



■ 정수 자료형

코드 2-24 오버플로

/2장/Variables

```
01 static void Main(string[] args)
02 {
03     int a = 2147483640;
04     int b = 52273;
05     Console.WriteLine(a + b);
06 }
```

- 현실 세계의 수학이라면 21475359130이 나오지만 C#은 음수를 출력

실행 결과

-2147431383

■ 정수 자료형

- 2,147,483,647에 1을 더하면 int 자료형의 범위를 넘어버리면서
-2,147,483,648이 됨
- 이러한 현상을 오버플로(Overflow)라고 부름

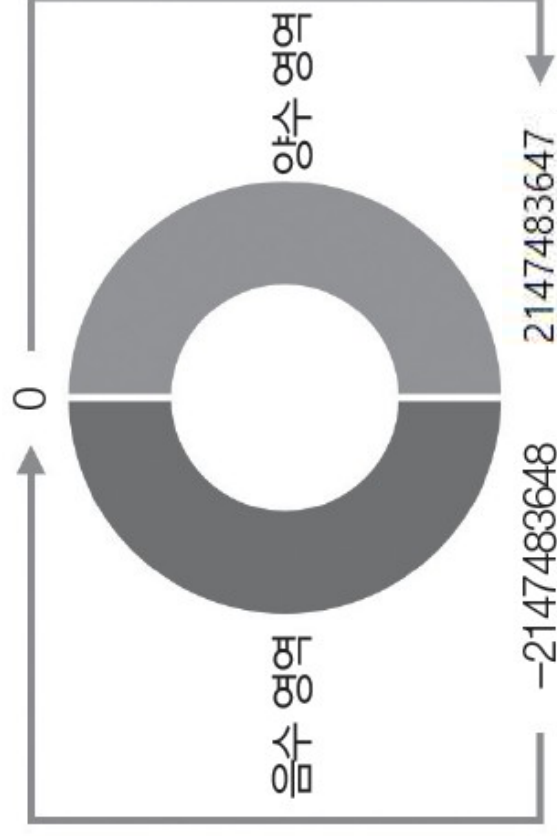


그림 2-12 int 자료형의 범위

■ 정수 자료형

■ 기본예제 2-12 오버플로

/2장/Overflow

코드 2-25 오버플로

```
01 static void Main(string[] args)
02 {
03     int a = 2000000000;
04     int b = 1000000000;
05     Console.WriteLine(a + b);
06 }
```

실행 결과

-1294967296

■ 정수 자료형

■ 기본예제 2-12 오버플로

/2장/Overflow

코드 2-26

자료형 변환을 사용한 해결 방법

```
01 static void Main(string[] args)
02 {
03     uint unsignedA = 2000000000;
04     uint unsignedB = 1000000000;
05     Console.WriteLine(unsignedA + unsignedB);
06 }
```

■ unsigned 자료형

- int 자료형은 -2,147,483,648부터 2,147,483,647까지 나타낼 수 있음
- 음수를 사용하지 않는다면 -2,147,483,648부터 -1까지의 숫자가 낭비됨
- 이런 낭비를 막고자 C#은 unsigned 자료형(부호가 없는 자료형)을 제공
- int와 long 키워드 대신 uint와 ulong 키워드를 사용

코드 2-27

uint와 ulong 자료형

/2장/Variables

```
01 static void Main(string[] args)
02 {
```

```
03     uint unsignedInt = 4147483647;
04     ulong unsignedLong = 11223372036854775808;
05
06     Console.WriteLine(unsignedInt);
07     Console.WriteLine(unsignedLong);
08 }
```

Microsoft Visual Studio 디버그 콘솔

4147483647
11223372036854775808

■ MaxValue와 MinValue

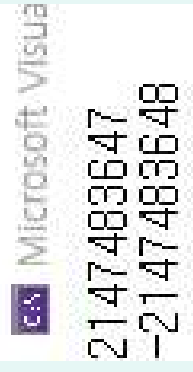
- C#은 `int.MaxValue`와 `int.MinValue`라는 코드로 손쉽게 최댓값과 최솟값을 알아낼 수 있음

코드 2-28

int 자료형의 최댓값과 최솟값

/2장/Variables

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(int.MaxValue);
04     Console.WriteLine(int.MinValue);
05 }
```



Microsoft Visual Studio
2147483647
-2147483648

■ MaxValue와 MinValue

- long 자료형의 최댓값과 최솟값은 다음과 같은 방법으로 알아낼 수 있음

코드 2-29

long 자료형의 최댓값과 최솟값

/2장/Variables

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(long.MaxValue);
04     Console.WriteLine(long.MinValue);
05 }
```

Microsoft Visual Studio 디버깅

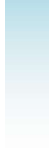
9223372036854775807
-9223372036854775808

■ 실수 자료형

- 실수 변수를 만들 때는 [표 2-14]의 자료형을 사용

표 2-14 실수 자료형

키워드	설명
float	4바이트의 실수
double	8바이트의 실수



■ 실수 자료형

■ 기본예제 2-13 실수 변수 생성

/2장/RealNumberVariable

코드 2-30 실수 변수 생성

```
01 static void Main(string[] args)
02 {
03     double a = 52.273;
04     double b = 103.32;
05
06     Console.WriteLine(a + b);
07     Console.WriteLine(a - b);
08     Console.WriteLine(a * b);
09     Console.WriteLine(a / b);
10 }
```

실행 결과

```
155.593
-51.047
5400.84636
0.50593302361595
```

■ 문자 자료형

- 문자 변수를 만들 때는 [표 2-15]의 자료형을 사용

표 2-15 문자 자료형

키워드	설명
char	문자



■ 문자 자료형

■ 기본예제 2-14 문자 변수 생성

/2장/CharacterVariable

코드 2-31

문자 변수 생성

```
01 static void Main(string[] args)
02 {
03     char a = 'a';
04     Console.WriteLine(a);
05 }
```

실행 결과

a

- **sizeof 연산자와 char 자료형의 크기**
- C#에서 char 자료형은 2바이트(sizeof 연산자로 확인)

sizeof(자료형)

그림 2-13 sizeof 연산자 형태

코드 2-32 sizeof 연산자

/2장/Variables

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("int: " + sizeof(int));
04     Console.WriteLine("long: " + sizeof(long));
05     Console.WriteLine("float: " + sizeof(float));
06     Console.WriteLine("double: " + sizeof(double));
07     Console.WriteLine("char: " + sizeof(char));
08 }
```

실행 결과

```
int: 4
long: 8
float: 4
double: 8
char: 2
```

■ 문자 자료형과 연산자

- 문자 자료형은 문자열 자료형보다 정수 자료형에 가까움

코드 2-33

문자 자료형과 연산자

/2장/Variables

```
01 static void Main(string[] args)
02 {
03     char a = 'a'; //아스키 코드: 97
04     char b = 'b'; //아스키 코드: 98
05
06     Console.WriteLine(a + b);
07     Console.WriteLine(a - b);
08     Console.WriteLine(a * b);
09     Console.WriteLine(a / b);
10     Console.WriteLine(a % b);
11 }
```

아스키코드

097	a
098	b
099	c
100	d

실행 결과

```
195
-1
9506
0
97
```

■ 문자열 자료형

- 문자열 변수를 선언할 때는 [표 2-16]의 자료형을 사용

표 2-16 문자열 자료형

키워드	설명
string	문자열 자료형



■ 문자 자료형

■ 기본예제 2-15 문자열 변수 생성 /2장/StringVariable

코드 2-34

문자열 변수 생성

```
01 static void Main(string[] args)
02 {
03     string message = "안녕하세요";
04
05     Console.WriteLine(message + "!");
06     Console.WriteLine(message[0]);
07     Console.WriteLine(message[1]);
08     Console.WriteLine(message[2]);
09 }
```

실행 결과

안녕하세요!

안
녕
하

■ sizeof 연산자와 string 자료형

- string 자료형은 sizeof 연산자로 자료형의 크기를 구할 수 없음
- 다음 코드를 실행하면 컴파일 오류가 발생

코드 2-35

sizeof 연산자와 string 자료형

/2장/Variables

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("string: " + sizeof(string));
04 }
```

- 문자열은 다른 기본 자료형과 다르기 때문
- 비주얼 스튜디오에서 자료형 위에 마우스를 올려놓으면 자료형의 진짜 원형을 찾을 수 있음

```
static void Main(string[] args)
{
    int output = 0;
}
```

readonly struct System.Int32
Represents a 32-bit signed integer.

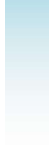
그림 2-14 int 자료형의 원형은 struct System.Int32 구조체

■ sizeof 연산자와 string 자료형

- 기본 자료형들의 원형은 [표 2-17]과 같음

표 2-17 기본 자료형의 원형

기본 자료형	원형	기본 자료형	원형
int	struct System.Int32	float	struct System.Single
long	struct System.Int64	double	struct System.Double
char	struct System.Char	bool	struct System.Boolean
string	class System.String		

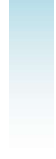


■ 불 자료형

- 불 변수를 선언할 때는 [표 2-18]의 자료형을 사용

표 2-18 불 자료형

키워드		설명
bool		불 자료형



■ 불 자료형

- 기본예제 2-16 불 변수 생성 /2장/BoolVariable

코드 2-36

불 변수 생성

```
01 static void Main(string[] args)
02 {
03     bool one = 10 < 0;
04     bool other = 20 > 100;
05
06     Console.WriteLine(one);
07     Console.WriteLine(other);
08 }
```

실행 결과

```
False
False
```